ローカルH2接続初期化

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-107
画面名	ローカルH2接続初期化
会社名	
作成日	

概要

概要説明

ローカルH2接続初期化の入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	N2_SIGHT.B顧客基本表、受給有無テーブル、年金1_LK、年金種類テーブル、年金__削除明細、店名テーブル、性別テーブル、裁定請求テーブル

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

contextInitialized

処理概要

ローカルH2接続初期化で「contextInitialized」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理12：詳細仕様参照）。

2. 「!Boolean.parseBoolean(System.getProperty(LOCAL_DATABASE_PROPERTY, "いいえ"))」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

2.2. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.1. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

3.2. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

3.3. 通常処理：「条件16を満たす（原本条件は補足説明を参照）、かつ条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

3.5. 通常処理：「対象項目」を設定する。

3.6. 通常処理：「対象項目」を設定する。

3.7. 通常処理：「対象項目」を設定する。

3.8. 通常処理：関連情報を取得・設定する（処理19：詳細仕様参照）。

3.9. 通常処理：関連情報を取得・設定する（処理20：詳細仕様参照）。

3.10. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

3-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

3-例外1.1. 例外発生時：関連情報を取得・設定する（処理22：詳細仕様参照）。

3-例外1.2. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-19BE5BBBA554

情報	内容
操作ID	SQL-19BE5BBBA554
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:628 / exists@623

結果・例外時の扱い

到達条件	結果
!Boolean.parseBoolean(System.getProperty(LOCAL_DATABASE_PROPERTY, "いいえ"))	処理結果を呼出元へ返す
NOT (!Boolean.parseBoolean(System.getProperty(LOCAL_DATABASE_PROPERTY, "いいえ"))) ・条件23を満たす（原本条件は補足説明を参照）、かつ条件24を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (!Boolean.parseBoolean(System.getProperty(LOCAL_DATABASE_PROPERTY, "いいえ"))) ・条件25を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

contextDestroyed

処理概要

ローカルH2接続初期化で「contextDestroyed」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理3：詳細仕様参照）。
2. 関連情報を取得・設定する（処理4：詳細仕様参照）。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す

resolveDatabaseFile

処理概要

ローカルH2接続初期化で「resolveDatabaseFile」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 「対象項目がある、かつ条件5を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がある、かつ条件7を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (overridePath が異なる 未設定 && overridePath.trim().length() > 0)・対象項目がない	処理の異常を通知し、この経路を終了する
NOT (overridePath が異なる 未設定 && overridePath.trim().length() > 0)・NOT (directory が等しい 未設定)	処理結果を呼出元へ返す

createJdbcUrl

処理概要

ローカルH2接続初期化で「createJdbcUrl」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

initializeDatabase

処理概要

ローカルH2接続初期化で「initializeDatabase」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

2.3. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

2-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

2-後処理.1. 終了時の共通後処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2-後処理.1.1. 条件成立時：使用した接続・処理資源を解放する。

runScript

処理概要

ローカルH2接続初期化で「runScript」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

4.2. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

4-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

4-後処理.1. 終了時の共通後処理：「対象項目がある」に該当するか確認する。成立時と不成立時の手順を分けて行う。

4-後処理.1.1. 条件成立時：使用した接続・処理資源を解放する。

4-後処理.1-別.1. 条件不成立時：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理の異常を通知し、この経路を終了する

bindDataSource

処理概要

ローカルH2接続初期化で「bindDataSource」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

2-例外1. 例外条件「NameAlreadyBoundException e」に該当した場合に次の手順を行う。型名は原本の表記。

1. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理8：詳細仕様参照）。

unbindDataSource

処理概要

ローカルH2接続初期化で「unbindDataSource」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理4：詳細仕様参照）。

1.2. 通常処理：繰返し条件「条件5を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

1.2.1. 繰返しの各回：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.2.1.1. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

1.2.1-例外1. 例外条件「NamingException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1. 例外条件「NamingException e」に該当した場合に次の手順を行う。型名は原本の表記。

shutdownDatabase

処理概要

ローカルH2接続初期化で「shutdownDatabase」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理9：詳細仕様参照）。

4.2. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

4.3. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

4-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：関連情報を取得・設定する（処理12：詳細仕様参照）。

4-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

4-後処理.1. 終了時の共通後処理：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4-後処理.1.1. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4-後処理.1.1.1. 条件成立時：使用した接続・処理資源を解放する。

4-後処理.1-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

4-後処理.2. 終了時の共通後処理：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4-後処理.2.1. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4-後処理.2.1.1. 条件成立時：使用した接続・処理資源を解放する。

4-後処理.2-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す

DB関連処理

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
contextInitialized	処理1	context.log("N2 pension compatibility database initialization failed.", e)
contextInitialized	処理2	context.log("N2 pension compatibility database initialized: " + databaseFile.getAbsolutePath())
contextInitialized	処理3	bindDataSource()

イベント	参照番号	原本条件・処理
contextInitialized	処理4	initializeDatabase(context)
contextInitialized	処理5	dataSource = new JdbcDataSource()
contextInitialized	条件6	!directory.exists()
contextInitialized	条件7	!directory.mkdirs()
contextInitialized	処理8	File directory = databaseFile.getParentFile()
contextInitialized	処理9	File databaseFile = resolveDatabaseFile(context)
contextInitialized	処理10	context.log("Local H2 compatibility database is disabled; using the N2 container JNDI database.")
contextInitialized	処理11	ServletContext context = event.getServletContext()
contextInitialized	処理12	ServletContext context = event.getServletContext()
contextInitialized	処理13	context.log("Local H2 compatibility database is disabled; using the N2 container JNDI database.")
contextInitialized	処理14	File databaseFile = resolveDatabaseFile(context)
contextInitialized	処理15	File directory = databaseFile.getParentFile()
contextInitialized	条件16	!directory.exists()
contextInitialized	条件17	!directory.mkdirs()
contextInitialized	処理18	dataSource = new JdbcDataSource()
contextInitialized	処理19	initializeDatabase(context)
contextInitialized	処理20	bindDataSource()
contextInitialized	処理21	context.log("N2 pension compatibility database initialized: " + databaseFile.getAbsolutePath())

イベント	参照番号	原本条件・処理
contextInitialized	処理22	context.log("N2 pension compatibility database initialization failed.", e)
contextInitialized	条件23	!directory.exists()
contextInitialized	条件24	!directory.mkdirs()
contextInitialized	条件25	Exception e
contextDestroyed	処理1	shutdownDatabase(event.getServletContext())
contextDestroyed	処理2	unbindDataSource()
contextDestroyed	処理3	unbindDataSource()
contextDestroyed	処理4	shutdownDatabase(event.getServletContext())
resolveDatabaseFile	処理1	String directory = context.getRealPath("/WEB-INF/local-db")
resolveDatabaseFile	条件2	overridePath.trim().length() > 0
resolveDatabaseFile	処理3	String overridePath = System.getProperty("n2nenkin.db.path")
resolveDatabaseFile	処理4	String overridePath = System.getProperty("n2nenkin.db.path")
resolveDatabaseFile	条件5	overridePath.trim().length() > 0
resolveDatabaseFile	処理6	String directory = context.getRealPath("/WEB-INF/local-db")
resolveDatabaseFile	条件7	overridePath.trim().length() > 0
initializeDatabase	処理1	runScript(context, connection, "/WEB-INF/resources/db/data.sql")
initializeDatabase	処理2	runScript(context, connection, "/WEB-INF/resources/db/schema.sql")
initializeDatabase	処理3	connection = dataSource.getConnection()

イベント	参照番号	原本条件・処理
initializeDatabase	処理4	Connection connection = null
initializeDatabase	処理5	Connection connection = null
initializeDatabase	処理6	connection = dataSource.getConnection()
initializeDatabase	処理7	runScript(context, connection, "/WEB-INF/resources/db/schema.sql")
initializeDatabase	処理8	runScript(context, connection, "/WEB-INF/resources/db/data.sql")
runScript	処理1	RunScript.execute(connection, reader)
runScript	処理2	reader = new InputStreamReader(stream, "UTF-8")
runScript	処理3	Reader reader = null
runScript	処理4	InputStream stream = context.getResourceAsStream(path)
runScript	処理5	InputStream stream = context.getResourceAsStream(path)
runScript	処理6	Reader reader = null
runScript	処理7	reader = new InputStreamReader(stream, "UTF-8")
runScript	処理8	RunScript.execute(connection, reader)
bindDataSource	条件1	int i = 0; i < JNDI_NAMES.length; i++
bindDataSource	処理2	context.rebind(JNDI_NAMES[i], dataSource)
bindDataSource	処理3	context.createSubcontext("jdbc")
bindDataSource	処理4	InitialContext context = new InitialContext()
bindDataSource	処理5	InitialContext context = new InitialContext()
bindDataSource	処理6	context.createSubcontext("jdbc")

イベント	参照番号	原本条件・処理
bindDataSource	条件7	int i = 0; i < JNDI_NAMES.length; i++
bindDataSource	処理8	context.rebind(JNDI_NAMES[i], dataSource)
unbindDataSource	条件1	int i = 0; i < JNDI_NAMES.length; i++
unbindDataSource	処理2	context.unbind(JNDI_NAMES[i])
unbindDataSource	処理3	Context context = new InitialContext()
unbindDataSource	処理4	Context context = new InitialContext()
unbindDataSource	条件5	int i = 0; i < JNDI_NAMES.length; i++
unbindDataSource	処理6	context.unbind(JNDI_NAMES[i])
shutdownDatabase	処理1	context.log("H2 shutdown failed.", e)
shutdownDatabase	処理2	statement.execute("SHUTDOWN")
shutdownDatabase	処理3	statement = connection.createStatement()
shutdownDatabase	処理4	connection = dataSource.getConnection()
shutdownDatabase	処理5	Statement statement = null
shutdownDatabase	処理6	Connection connection = null
shutdownDatabase	処理7	Connection connection = null
shutdownDatabase	処理8	Statement statement = null
shutdownDatabase	処理9	connection = dataSource.getConnection()
shutdownDatabase	処理10	statement = connection.createStatement()
shutdownDatabase	処理11	statement.execute("SHUTDOWN")

イベント	参照番号	原本条件・処理
shutdownDatabase	処理12	context.log("H2 shutdown failed.", e)